A Minimalist Putback-Based Bidirectional Programming Language, Formally Verified

Hsiang-Shang Ko[2], Tao Zan[1], and Zhenjiang Hu[1,2]

[1] SOKENDAI (The Graduate University for Advanced Studies), Japan
{taozan,hu}@@nii.ac.jp
[2] National Institute of Informatics, Japan
hsiang-shang@@nii.ac.jp

1 Introduction

Due to the increase in technology, smart devices for different purpose are developed, thus come into an explosion in the number of devices and information. The duplication of information among different devices makes it hard to be kept consistent which is an important issue. Bidirectional transformation (BX) [3] consists a pair of *get* which extracts information from a source to construct an abstract view and *put* which propagates updates on view back to the source , that provides a novel mechanism for maintaining the consistency between two related information (source and view). Many bidirectional programming languages [1,2,5–12] are designed to aid user to write bidirectional transformations, thus user only needs to write one program that is interpreted in two directions. The correctness of this BX program is guaranteed by well-behavedness [5]. The existing BXs are *get*-based, which let user to write the BX program in the direction of *get* to describe how to construct the abstract view form source, and system derives a suitable backward *put* for free. While in fact there are usually more than one correct *put* for the *get*, and *get*-based BX only provides a default or limited strategies which hinder BX from real practical usage. Recently, a new approach called putback-based bidirectional transformation [4] was proposed which designs the BX from *put* instead of *get* which declares that it lets user have full control over the *put* to specify the update policies and an unique and correct forward transformation *get* is derived for free. Typically, a combinator library *putlenses* [13] provides a set of basic lens combinators to let user write complex bx programs in the putback-based way, BiFlux [14] is a bidirectional function update language for XML which is a carefully designed putback-based bx language. The update program user written using BiFluX in fact is the *put* function, and the system derives *get* for free.

The pioneer work of putback-based BX gives opportunities to make BX really useful in real applications as it lets user to have full control over the update strategy. While the well-behavedness is not fully formal verified for both *putlenses* and BiFluX which is the most important property for BX. As the forward transformation *get* is derived for free and user do not need to care how *get* is derived and not have to check the derived *get* is correct or not manually, it is necessary to fully guarantee the program user written is well-behaved by

the BX language. For example, in BiFluX, the semantics of the basic update operator *replace* is straight-forward and the well-beahvedness for *replace* is not checked, and complex ones like if statement are checked dynamically for the well-behavednss while all of them are not formally verified. Since BiFluX is designed for XML data, it needs to handle complex regular expressions and XML structured data which makes the program hard to check.

It turned out that a small but powerful putback-based core language which is fully verified to guarantee the well-behavedness is needed to serve as the base for all high-level putback-based languages. Thus we design and implement BiGUL, a minimalist datatype-generic putback-based language. We design BiGUL to be minimal to make it easy for usage and property checking. For example, there is no if statement in BiGUL as it can be expressed as a special case of *case* statement. While the well-behavedness of BiGUL is fully verified using Agda language.

Our contribution can be summarised as follows:

- we propose a minimalist putback-based bidirectional language BiGUL as a core language which is fully certified to guarantee the correctness, and served as a base for other putback-based bidirectional languages.
- A correct forward transformation *get* is automatically constructed through BiGUL programs. User concentrates on specifying *put*, our system guarantees the derived *get* together with the *put* is well-behaved.
- BiGUL has been implemented using Haskell and the well-behavedness has been proved using Agda language and all the source code are available [3].

Theory construction in terms of Agda programming

2 Discussion

recursion (the lack thereof)

extensions: for example, iteration, view dependency

Haskell implementation

Towards a dependently typed version

stems from work on BiFluX; comparison with putlenses

Theory construction in terms of Agda programming
Regarding formalisation: "[W]e are interested in extending what can be calculated precisely because we are not interested in the calculations themselves. Developing techniques that allow more of a derivation to be calculated allows attention to be focussed on the creative parts, which cannot be calculated." [**?**]
[**?**]

[3] http://www.prg.nii.ac.jp/projects/bigul

References

1. Davi M. J. Barbosa, Julien Cretin, Nate Foster, Michael Greenberg, and Benjamin C. Pierce. Matching lenses: alignment and view update. In *International Conference on Functional Programming*, pages 193–204. ACM, 2010.
2. A. Bohannon, B. C. Pierce, and J. A. Vaughan. Relational lenses: a language for updatable views. In *Principles of Database Systems*, pages 338–347. ACM, 2006.
3. K. Czarnecki, J. N. Foster, Z. Hu, R. Lämmel, A. Schürr, and J. Terwilliger. Bidirectional transformations: A cross-discipline perspective. In *International Conference on Model Transformation*, volume 5563 of *Lecture Notes in Computer Science*, pages 260–283. Springer-Verlag, 2009.
4. Sebastian Fischer, Zhenjiang Hu, and Hugo Pacheco. The essence of bidirectional programming. *SCIENCE CHINA Information Sciences*, 58(5):1–21, 2015.
5. J. Nathan Foster, M. B. Greenwald, J. T. Moore, B. C. Pierce, and A. Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3):17, 2007.
6. J. Nathan Foster, Alexandre Pilkiewicz, and Benjamin C. Pierce. Quotient lenses. In *International Conference on Functional Programming*, pages 383–396. ACM, 2008.
7. S. Hidaka, Z. Hu, K. Inaba, H. Kato, K. Matsuda, and K. Nakano. Bidirectionalizing graph transformations. In *International Conference on Functional Programming*, pages 205–216. ACM, 2010.
8. M. Hofmann, B. C. Pierce, and D. Wagner. Symmetric lenses. In *Principles of Programming Languages*, pages 371–384. ACM, 2011.
9. M. Hofmann, Benjamin C. Pierce, and D. Wagner. Edit lenses. In *Principles of Programming Languages*, pages 495–508. ACM, 2012.
10. Zhenjiang Hu, Shin-Cheng Mu, and Masato Takeichi. A programmable editor for developing structured documents based on bidirectional transformations. *Higher-Order and Symbolic Computation*, 21(1-2):89–118, 2008.
11. Kazutaka Matsuda, Zhenjiang Hu, Keisuke Nakano, Makoto Hamana, and Masato Takeichi. Bidirectionalization transformation based on automatic derivation of view complement functions. In *International Conference on Functional Programming*, pages 47–58. ACM, 2007.
12. H. Pacheco and A. Cunha. Generic point-free lenses. In *Mathematics of Program Construction*, volume 6120 of *Lecture Notes in Computer Science*, pages 331–352. Springer-Verlag, 2010.
13. Hugo Pacheco, Zhenjiang Hu, and Sebastian Fischer. Monadic combinators for "putback" style bidirectional programming. In *Partial Evaluation and Program Manipulation*, pages 39–50. ACM, 2014.
14. Hugo Pacheco, Tao Zan, and Zhenjiang Hu. BiFluX: A bidirectional functional update language for XML. In *Principles and Practice of Declarative Programming*, pages 147–158. ACM, 2014.